Handle Events in Lightning Web Components

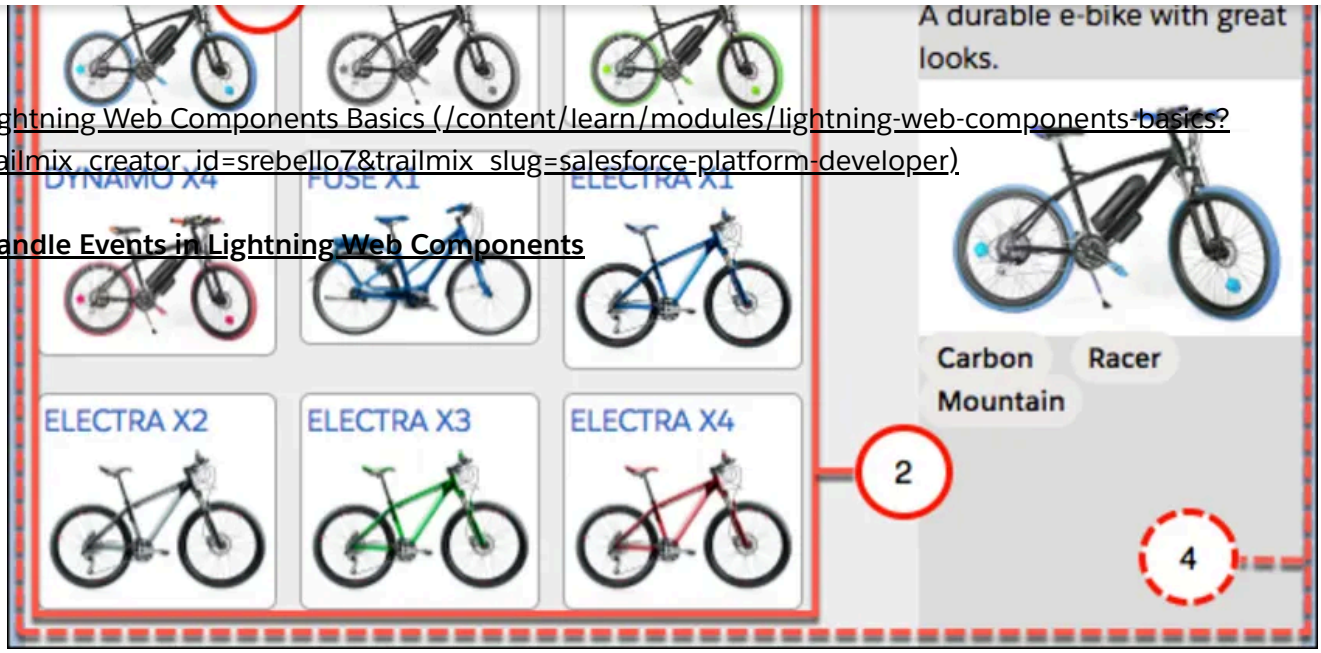
Learning Objectives

After completing this unit, you'll be able to:

- Create an app that includes multiple components.
- Describe the file structure of a complex component.
- Handle events.

Follow an Event's Journey

You've built a component and pushed it to an org. Let's start adding some interactivity with event handling. We follow an event's journey through several components for sophisticated event handling in an application. This application is a product selector for a bicycle shop. Users click a bike name and image to see more detail.



Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

This app has four components working together.

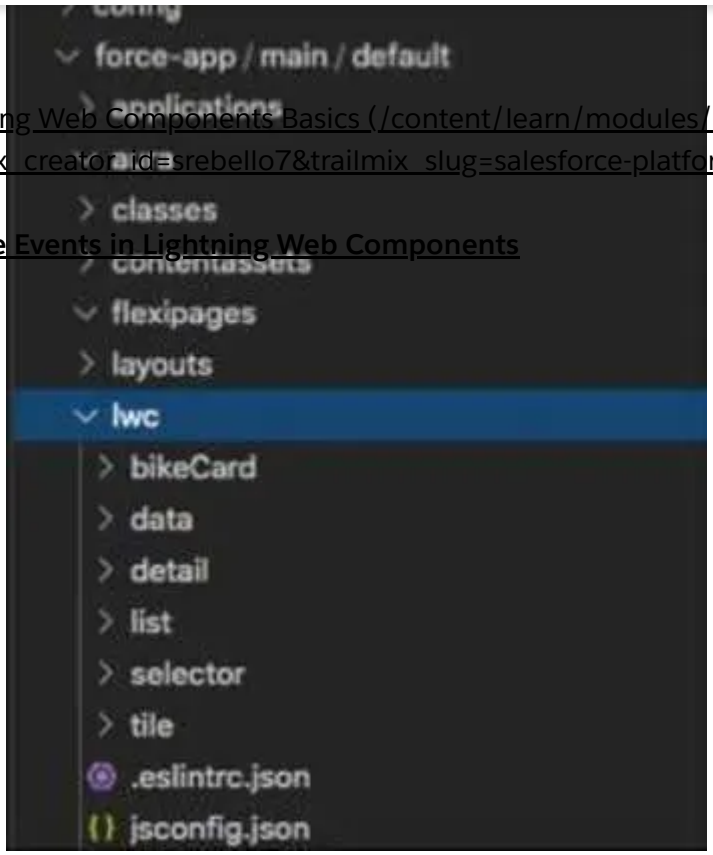
1. **tile**: displays an individual item.
2. **list**: arranges the tiles.
3. **detail**: displays item details when a tile is clicked (similar to the bikeCard you just created).
4. **selector**: contains the whole set of components. A container component isn't required, but we're using one here to help with event handling.

For now, the application uses a data file to load static data for testing. In the next unit, you learn how to pull dynamic data from an org.

Component Composition

Let's add some files to our project that we can deploy to an org.

1. Download the files for this app here: Bike Selector App for Trailhead (https://dfc-data-production.s3.amazonaws.com/files/LWC-Basics-Bike-Selector-Trailhead.zip).



Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

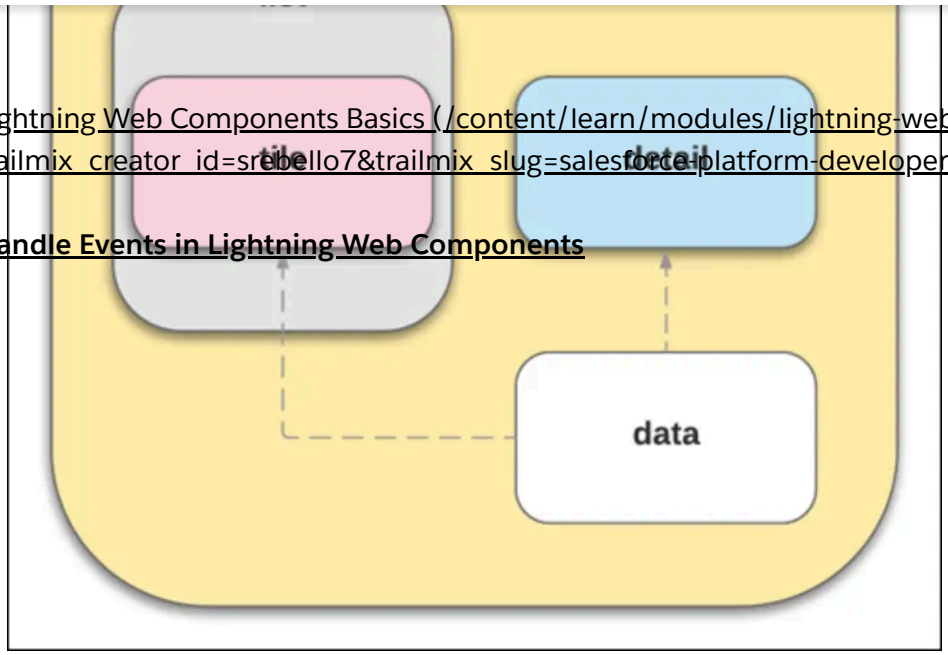
Handle Events in Lightning Web Components

Component Relationships

In this app, multiple components work together; some components are nested inside other components. Just as you nest HTML elements inside each other, Lightning web components–which are custom HTML elements–can be nested inside other Lightning web components.

In our file system, the components' folders don't really give insight into the relationships between them.

Let's look at how the components nested at the UI level in a diagram.



Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

By looking at the files you can see the selector component lays out the page and renders the list(`c-list` copy) and detail(`c-detail` copy) components.

```
<template>
  <div class="wrapper">
    <header class="header">Select a Bike</header>
    <section class="content">
      <div class="columns">
        <main class="main" >
          <c-list onproductselected={handleProductSelected}></c-
list>
        </main>
        <aside class="sidebar-second">
          <c-detail product-id={selectedProductId}></c-detail>
        </aside>
      </div>
    </section>
  </div>
</template>
```

Update detail.html with the following:

```
<template>
  <template lwc:if={product}>

```

```
{product.fields.Description__c.value}</div>
      <img class="product-img" src=
{product.fields.PictureURL__c.value}></li>
{product.fields.Name.value}</p>
      <p>
        <lightning-badge label=
{product.fields.Material__c.value}></lightning-badge>
        <lightning-badge label={product.fields.Level__c.value}>
</lightning-badge>
      </p>
      <p>
        <lightning-badge label=
{product.fields.Category__c.value}></lightning-badge>
      </p>
```

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

Looking in detail.html, you see conditional rendering(`lwc:if={product}` copy and `lwc:else` copy). If nothing has been chosen from the list, then what's displayed is a message asking the user to choose something. If something is chosen, it displays the bike information.

The list component renders several tile(`c-tile` copy) components, one for each bike in the data. This nesting is achieved in the HTML for each parent component. For example, the list component has the following HTML, including the tile component as `c-tile` copy .

```
<template>
  <div class="container">
    <template for:each={bikes} for:item="bike">
      <c-tile key={bike.fields.Id.value} product={bike}
ontileclick={handleTileClick}></c-tile>
    </template>
  </div>
</template>
```

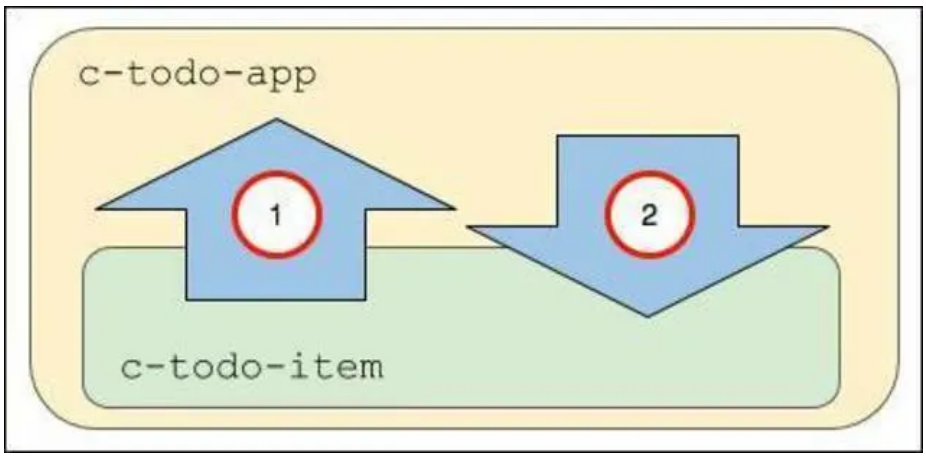
Notice how each iteration of the bike item spawns a new tile component. Simply including the `c-tile` copy component tag makes each tile component its child. The div class definition “container” is used for styling, so you can control the arrangement of tiles. If you look at list.css, you see that it wraps the content.

for event handling.

Let's dive a little deeper into event handling.

Events Up, Properties Down

In a complex component (one that contains several parent and child components), the components can communicate up and down.



1. The c-todo-item child component dispatches an event to the parent c-todo-app component. For example, the child can pass an event object to the parent when a user clicks a button so the parent can handle the event and change the current page.
2. The c-todo-app parent component passes a property or invokes a method in the child component. For example, the parent can set a text value in a child component, or invoke a method in the child component.

Let's look at how this communication works.

Passing Information Up

Information can be passed up using events (https://developer.salesforce.com/docs/component-library/documentation/en/lwc/lwc.events_create_dispatch) and event listeners (https://developer.salesforce.com/docs/component-library/documentation/en/lwc/lwc.events_handling).

The child component dispatches the event and the parent component listens for it. Dispatching the event includes creating an event object the child can pass to the parent component. The parent has a handler to respond to the event.

```
// todoItem.js
import { LightningElement } from 'lwc';
export default class TodoItem extends LightningElement {
    nextHandler() {
        this.dispatchEvent(new CustomEvent('next'));
    }
}
```

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

Event types can be any string but should conform to the DOM event standard of no uppercase letters, no spaces, and use underscores to separate words if necessary.

The parent component listens for the event with the inline event handler prefixed with 'on'(onnext).

```
<!-- todoApp.html -->
<template>
  <c-todo-item onnext={nextHandler}></c-todo-item>
</template>
```

And passes the event object to an event handler.

```
// todoApp.js
import { LightningElement } from 'lwc';
export default class TodoApp extends LightningElement {
  ...
  nextHandler(){
    this.page = this.page + 1;
  }
}
```

Passing Information Down

Information can be passed down using public properties (https://developer.salesforce.com/docs/component-library/documentation/en/lwc/lwc.create_components_data_binding) and public

For example (don't create these components), if the c-todo-item child component has the following:

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

```
// todoItem.js
import { LightningElement, api } from 'lwc';
export default class TodoItem extends LightningElement {
  @api itemName;
}
```

Set the value from the parent with the following:

```
<!-- todoApp.html -->
<template>
  <c-todo-item item-name="Milk"></c-todo-item>
</template>
```

Notice that the `itemName` copy variable gets set using the kebab case attribute `item-name` copy. Property names in JavaScript are in camel case while HTML attribute names are in kebab case (dash-separated) to match HTML standards. The `item-name` copy attribute in markup maps to the `itemName` copy JavaScript property.

Public properties are great solutions for passing down primitive values, simple objects, and arrays.

Also, you can use getters & setters (https://developer.salesforce.com/docs/component-library/documentation/en/lwc/lwc.js_props_getters_setters) to execute some logic when properties are get or set. And remember, annotate them with the `@api` copy decorator to make them public for other components.

Similarly, you can create public methods that are callable from a parent component. Create a public method in the child component by defining it with the `@api` copy decorator, then call it from the parent component.

Let's say that we have a child component like this one (don't create these components).

```
// videoPlayer.js
import { LightningElement, api } from 'lwc';
export default class VideoPlayer extends LightningElement {

```

When the c-video-player component is included in a parent component, we can invoke the method from the parent component like this:

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

```
// methodCaller.js
import { LightningElement } from 'lwc';
export default class MethodCaller extends LightningElement {
  handlePlay() {
    this.template.querySelector('c-video-player').play();
  }
}
```

We defined a method `handlePlay()` copy that fires the event. Then we use the `querySelector()` copy DOM method to search for a DOM element called c-video-player and invoke its public method.

Handling Events in HTML

So our selector app needs to handle one type of event–the user clicking a tile. When this happens, the detail component should re-render with the information from the related tile. You can handle events in HTML (add an event listener in the template) or JavaScript (write an event listener function). We recommend using the HTML approach, as follows.

Each tile component listens for the user click because the tile component's HTML (tile.html) contains an `onclick` copy event listener.

```
<template>
  <div class="container">
    <a onclick={tileClick}>
      <div class="title">{product.fields.Name.value}</div>
      <img class="product-img" src=
{product.fields.Picture_URL__c.value} alt=
{product.fields.Name.value}/>
    </a>
  </div>
</template>
```

When a user clicks one of the tile instances in the UI, the `onclick` copy listener calls the handler function `tileClick` copy in the tile.js JavaScript file.

```
        // detail contains only primitives
        detail: this.product.fields.Id.value
    });
    // Fire the event from c-tile
    this.dispatchEvent(event);
  }
}
```

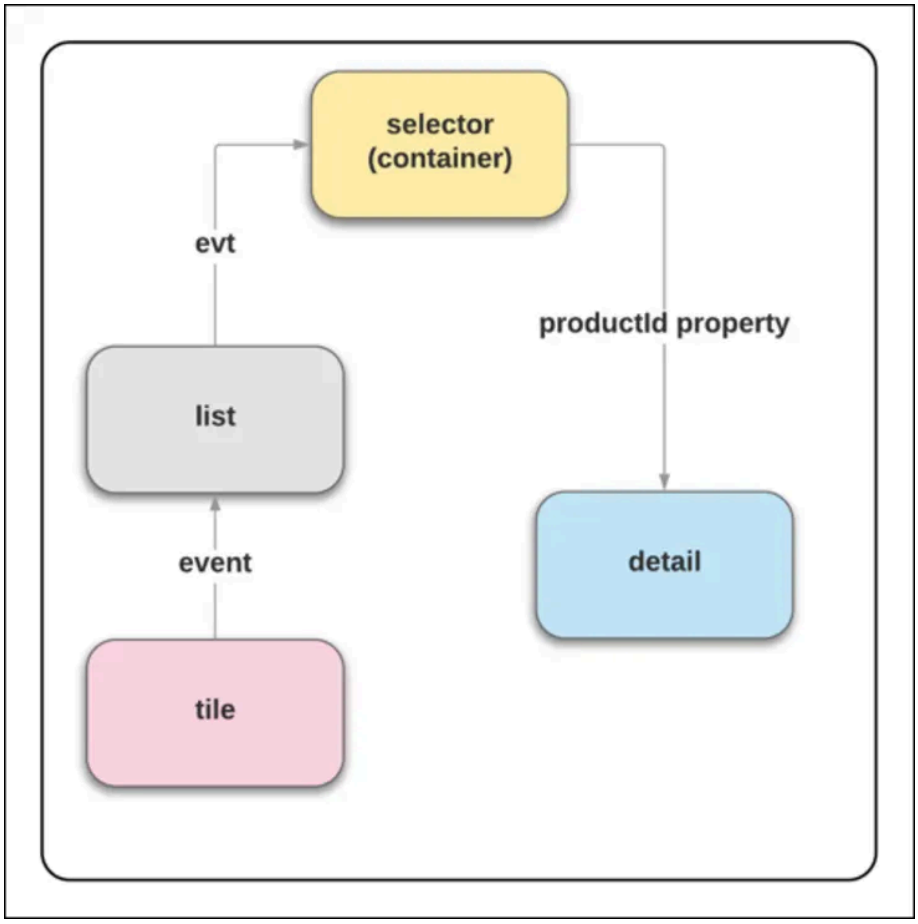
Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

Selector App Event Pattern

In our product selector app, we use a complex component (one that contains several parent and child components). We recommend that you propagate the event up through the component hierarchy, so parent components can respond to child events. If you have other child components (not the one firing the event), you can pass a property down to those children in response to the event.

The pattern looks like this:



To do this, we need to chain event listeners and handlers up the hierarchy to the ebikes component. Then pass a property down to the detail component.

event type `tileclick` copy and an object containing a `detail` copy value (`this.product.fields.Id.value` copy).

3. list.js has the `handleTileClick` copy listener that calls the `handleProductClick` copy handler.

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

4. list.js has the `handleTileClick` copy method that passes in the event(`evt` copy) to create another `CustomEvent` copy (`productselected` copy) with an object also containing a `detail` copy value `evt.detail` copy. And it dispatches the event in JavaScript:

Handle Events in Lightning Web Components

```
// Fire the event from c-list
this.dispatchEvent(event);
```

5. selector.html has the `onproductselected` copy event listener that calls the `handleProductSelected` copy handler.
6. selector.js has the `handleProductSelected` copy method set `selectedProductId` copy to the `evt.detail` copy value that was passed into it. The variable "selectedProductId" is passed from the selector component to the detail component in selector.html:
   `product-id={selectedProductId}` copy .
7. detail.html has a conditional directive (remember those from Unit 2?) waiting for a product value:
   `<template lwc:if={product}>` copy
8. detail.js brings the pieces together. It creates a private variable `_productId` copy to track the state of the `productId` copy value. Then it uses a get/set pattern to get the value and set it to a variable `product` copy which lets detail.html load the conditional content.

Getters and setters are a common JavaScript construction. They allow you to add logic and conditions to property assignments.

```
import { LightningElement, api } from 'lwc';
import { bikes } from 'c/data';
export default class Detail extends LightningElement {
  product;
  // Private var to track @api productId
  _productId = undefined;
  // Use set and get to process the value every time it's
  // requested while switching between products
```

```
    // getter for productId
    @api get productId(){
        return this._productId;
    }
}
```

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

Each time you click a tile, this process repeats itself.

Events have properties to manage the propagation of the event up the DOM tree. You can read more about them in Configure Event Propagation (https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.events_propagation). Changing the default values is for advanced event handling, and requires testing to ensure the expected behavior.

Deploy Your Files to Your Org

Let's deploy these new bikeCard project files to your org to see how it works. Using the same steps you performed in the last unit, deploy your new files, open the org, and create a page in Lightning App Builder with this app.

1. In the VS Code bikeCard project, right-click on the **force-app/main/default** folder and select **SFDX: Deploy Source to Org**.
2. From the Command Palette in VS Code, use **SFDX: Open Default Org** to open your org.
3. Create a one region page using the selector component.
4. Give it the label `Your Bike Selection` copy.
5. Drag your **selector** component to the top of the page layout.
6. Save and Activate for all users.
7. Open it and see your component working in the UI.

You've got a fully interactive page composed of several components working together. Next, we experiment with styling and getting live data from an org.

Resources

- *Lightning Web Components Developer Guide*: Shadow DOM (https://developer.salesforce.com/docs/component-

(https://developer.salesforce.com/docs/component-library/documentation/en/lwc/lwc.js_props_getters_setters)

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points

1 Which syntax correctly references a custom component?

A <c-detail product-id={selectedProductId}></c-detail>

B import { LightningComponent, api } from 'detail';

C import { detail, api } from 'lwc';

D

E <template if:true={product}>

2 Which of the following describes an event handling pattern in a complex component app?

A Properties are dispatched up from child to parent, then events are passed down from parent to child.

C passed down from parent to child.

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello72&trailmix_slug=salesforce-platform-developer)

D listen to the parent component.

Handle Events in Lightning Web Components

E Events are dispatched down from parent to all the child components.

Second attempt earns 50 points. Three or more earns 25 points.

Check the Quiz to Earn 100 Points

Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Handle Events in Lightning Web Components